## II. Approved Automotive Engineering AI Case Studies

Each student/team is allocated one of the five approved case studies below. The synopsis, implementation, technical report and demonstration must stay within the allocated case study's scope and governance principle.

| ID | Case Study | Project Overview | Expected Project Outputs |
|---|---|---|---|
| **CS1** | AUTOSAR HLD Document Analysis Assistant | Automate extraction, search and validation of architectural knowledge from AUTOSAR High-Level Design documents using RAG. | Component/interface inventory, dependency maps, cited Q&A responses, inconsistency report. |
| **CS2** | Functional Safety HARA and Safety Requirement Management Assistant | Guided, evidence-based support for hazard identification, ASIL assessment, safety requirement drafting and traceability (ISO 26262). | Candidate hazards/hazardous events, ASIL recommendation with rationale, draft safety goals/requirements, traceability matrix. |
| **CS3** | Automotive Cybersecurity TARA Assistant | Architecture-aware asset identification, attack-surface analysis, threat scenarios and mitigations (ISO/SAE 21434). | Asset inventory, attack-surface map, threat/damage scenarios, mitigation and cybersecurity requirement recommendations. |
| **CS4** | Secure Code Debugging and Review Assistant | Locally hosted AI assistance for code understanding, debugging, review and secure-coding checks without exposing source code. | Code/module summaries, candidate defects and root causes, MISRA-oriented review notes, structured finding reports. |
| **CS5** | UDS Diagnostics and Automated Test Generation Assistant | Interpret diagnostic specifications (ISO 14229), construct valid requests, and generate positive/negative test cases. | Request/response templates, positive and negative test scenarios, automation script templates (Python/CANoe/CAPL), coverage report. |

All five case studies share a common reference architecture: a locally or privately hosted pre-trained LLM, Retrieval-Augmented Generation with locally stored embeddings (FAISS/ChromaDB), a Python service layer (FastAPI/Flask), and a lightweight engineering UI (Streamlit). Outputs must remain evidence-based, cite their source, and be reviewed and approved by a human before use in compliance, design, release or test decisions.

### Detailed Requirements by Case Study

The following requirements apply in addition to the general submission requirements in Section III. Students must implement only the functional capabilities relevant to their allocated case study; the evaluation rubric is assessed against this scope, not against every capability listed for every case study in the brief.

#### CS1. AUTOSAR HLD Document Analysis Assistant

High-Level Design documents describe software components, interfaces, ports, signals, dependencies and functional flows, and are often hundreds of pages of unstructured PDF. Manual review makes it difficult to identify mismatches, missing dependencies, inconsistent terminology and incomplete coverage.

**Project objectives:**

- Reduce manual effort required to review HLD documents.

- Create a structured and searchable repository of architecture knowledge.
- Improve component, interface, signal and dependency traceability.
- Support design reviews, onboarding, impact analysis and test planning.

| In Scope | Out of Scope |
|---|---|
| PDF/document ingestion and OCR for scanned pages; Component, interface, port, signal and flow extraction; Natural-language search with citations; Document comparison and inconsistency reporting; Export of structured findings | Automatic approval of architecture decisions; Modification of source HLD documents without review; Replacement of formal architecture governance |

**Primary users / stakeholders:** System/AUTOSAR architects, Software developers, Integration engineers, Test engineers

**Required functional capabilities:**

- Document upload and controlled ingestion
- Section, table and metadata extraction
- Architecture entity recognition
- Interface and dependency mapping
- Natural-language question answering with citations
- Cited summaries and component reports
- Revision comparison
- Potential mismatch and completeness checks

| Inputs | Expected Outputs |
|---|---|
| Approved/synthetic AUTOSAR HLD documents; Interface specifications; Architecture descriptions; Component catalogues; Revision history | Component and interface inventory; Dependency maps; Functional-flow summaries; Searchable Q&A responses with citations; Potential inconsistency report; Structured export for downstream use |

**Mandatory controls:**

- Use only approved or synthetic documents; apply project isolation and access control.
- Restrict answers to retrieved evidence; show source citations and confidence/limitation indicators.
- Require human review before architecture or design decisions rely on generated output.

### CS2. Functional Safety HARA and Safety Requirement Management Assistant

Hazard Analysis and Risk Assessment under ISO 26262 depends on architecture, operational scenarios and expert judgment. Manual, spreadsheet-based execution can cause inconsistent hazard identification, variable severity/exposure/controllability (S/E/C) assessment, and fragmented traceability from hazards to safety goals, requirements, architecture and tests.

**Project objectives:**

- Improve consistency of hazard identification and ASIL assessment.
- Reduce repetitive HARA preparation effort.
- Maintain continuous traceability across safety work products.
- Improve audit readiness while retaining expert approval.

| In Scope | Out of Scope |
|---|---|
| Item definition and functional input analysis; Hazard and hazardous-event suggestions; Guided S, E and C assessment; Candidate ASIL recommendation with rationale; Safety goal and requirement drafting; Traceability and reporting | Autonomous safety approval; Replacement of certified safety professionals; Use of unapproved standards content |

**Primary users / stakeholders:** Functional safety engineers, System architects, Requirement engineers, Verification teams, Safety managers/assessors

**Required functional capabilities:**

- Guided HARA workflow
- Retrieval of similar approved hazards
- Rule-based ASIL decision support
- Drafting of safety goals, FSRs and TSRs
- Requirement quality checks
- Traceability matrix generation
- Decision history and audit evidence

| Inputs | Expected Outputs |
|---|---|
| Item definition; System architecture; Operational scenarios; Approved HARA libraries; Safety rules and templates; Verification artifacts | Candidate hazards and hazardous events; ASIL recommendation and rationale; Draft safety goals, FSRs and TSRs; Traceability matrix; Review and audit report |

**Mandatory controls:**

- Combine deterministic rules with cited evidence for ASIL rationale; do not rely on the LLM alone for the classification.
- Use only authorized internal/standard knowledge sources.
- Require mandatory human approval and decision logging before any safety artifact is treated as final.

### CS3. Automotive Cybersecurity TARA Assistant

Threat Analysis and Risk Assessment under ISO/SAE 21434 requires interpreting architecture, communication interfaces, data flows, assets and trust boundaries. Manual execution is expertise-intensive and prone to inconsistent threat coverage and mitigation selection across similar systems.

**Project objectives:**

- Improve asset and attack-surface coverage.
- Accelerate threat-scenario generation and risk analysis.
- Standardize mitigation recommendations.
- Create a reusable, controlled cybersecurity knowledge base.

| In Scope | Out of Scope |
|---|---|

| | |
|---|---|
| Architecture and interface analysis; Asset and trust-boundary identification; Threat and damage-scenario suggestions; Guided risk assessment; Mitigation and cybersecurity requirement recommendations; Traceability and reports | Autonomous cybersecurity acceptance; Active vulnerability exploitation; Replacement of qualified cybersecurity review |

**Primary users / stakeholders:** Cybersecurity engineers, System/network architects, Security testers, Software teams, Compliance/audit teams

**Required functional capabilities:**

- Asset extraction from architecture inputs
- CAN/Ethernet/diagnostics/OTA attack-surface mapping
- Threat-scenario recommendation from approved libraries
- Risk-assessment guidance
- Mitigation and requirement drafts
- Reusable threat library
- Traceability from assets to tests

| Inputs | Expected Outputs |
|---|---|
| System architecture; Communication matrix; Interface specifications; Asset catalogues; Approved threat libraries; Previous TARA artifacts | Asset inventory; Attack-surface map; Threat and damage scenarios; Risk-assessment support; Mitigation recommendations; Cybersecurity traceability report |

**Mandatory controls:**

- No active exploitation of vulnerabilities; use approved, isolated test/synthetic data only.
- Show source, assumptions and alternatives for every mitigation recommendation.
- Require expert (red-team) review and a coverage checklist before acceptance.

### CS4. Secure Code Debugging and Review Assistant

ECU software teams perform continuous debugging, peer review, MISRA-oriented analysis and security review. External AI tools may be restricted due to source-code confidentiality, IP and regulatory concerns, so the assistant must run locally or in a private environment without exposing code externally.

**Project objectives:**

- Reduce debugging and review effort.
- Improve consistency of review quality.
- Ground recommendations in internal coding rules and approved guidance.
- Keep source code and model interactions inside organizational boundaries.

| In Scope | Out of Scope |
|---|---|
| Code explanation and module summaries; Compiler, static-analysis and log interpretation; Candidate defect and root-cause suggestions; MISRA-oriented review assistance; Security-focused code review; Structured finding reports and repository integration | Automatic code merge or release approval; Replacement of qualified reviewers or certified tools; External transmission of source code |

**Primary users / stakeholders:** Embedded software developers, Code reviewers, Technical leads, Cybersecurity engineers, Quality managers

**Required functional capabilities:**

- Repository-aware code retrieval
- Function/module/dependency/control-flow summaries
- Analysis of compiler warnings and logs
- Candidate null-pointer, boundary, resource and state-issue detection
- Coding-standard explanation
- Secure-coding recommendations
- Structured findings with evidence and status

| Inputs | Expected Outputs |
|---|---|
| Approved/synthetic source code; Build logs; Compiler warnings; Static-analysis reports; Internal coding standards; Historical approved findings | Code summaries; Candidate root causes; Review findings; MISRA-oriented explanations; Security observations; Suggested fixes; Structured review report |

**Mandatory controls:**

- Run locally/privately with no external code transmission; encrypt and isolate the environment.
- Treat all repository content as untrusted text; separate instructions from content to prevent prompt injection.
- Require compilation, test and reviewer validation before any suggested fix is accepted.

### CS5. UDS Diagnostics and Automated Test Generation Assistant

UDS testing requires interpreting ISO 14229 together with OEM- and ECU-specific implementations, constructing valid requests, determining session/security prerequisites, and producing positive and negative test cases. The volume of services, identifiers and variants increases inconsistency and error risk when done manually.

**Project objectives:**

- Reduce diagnostic specification interpretation effort.
- Generate consistent positive and negative test scenarios.
- Validate request and expected-response structures.
- Accelerate conversion of requirements into executable test templates.

| In Scope | Out of Scope |
|---|---|
| Approved standard and OEM knowledge retrieval; UDS service and parameter guidance; Request-message construction; Positive and negative test generation; Expected-response validation; Export to Python, CANoe, CAPL or approved frameworks | Execution against production vehicles without controls; Use of unauthorized standards content; Unreviewed automatic acceptance of ECU behavior |

**Primary users / stakeholders:** Diagnostic test engineers, ECU software developers, Validation engineers, Test automation developers, Quality managers

**Required functional capabilities:**

- Natural-language diagnostic knowledge search
- Service/subfunction/DID/RID/NRC/session/security guidance
- Request-structure validation
- Test-case generation with preconditions and pass/fail criteria
- Positive and negative coverage generation
- Automation template export
- Response comparison and reporting

| Inputs | Expected Outputs |
|---|---|
| Authorized UDS specifications; OEM diagnostic specifications; ECU diagnostic extract; Requirements; Existing tests; Automation templates | Diagnostic guidance with citations; Validated request template; Expected positive response; Negative-response scenarios; Structured test case; Automation script template; Coverage report |

**Mandatory controls:**

- Apply deterministic protocol rules alongside generated guidance; require expert review of generated requests.
- Keep standard, OEM, ECU and project knowledge in separate, authorized collections.
- Execute generated tests only in a simulator or approved lab environment, never on production vehicles.

## III. Student Submission Requirements

### 1. Technical Artifacts

Each student/team must submit:

- Approved project synopsis with the faculty approval record.
- Source code / notebooks – data handling, model/RAG pipeline or agent logic, evaluation scripts.
- Approved or synthetic input data/knowledge base used by the solution.
- Pre-trained model and configuration details – model/provider, version, embeddings, vector store, prompts, tools.
- Evaluation evidence – test questions or ground-truth set, metrics, sample outputs with citations.

*Optional: additional automation scripts, helper utilities, or custom evaluation notebooks.*

### 2. Technical Report (PDF)

Must include:

- Problem statement and intended users
- Input data / knowledge-base description
- Solution architecture (pre-trained model, RAG pipeline and/or AI-agent design)
- Implementation and configuration explanation with workflow diagrams
- Evaluation steps and results
- Responsible AI measures adopted (grounding, privacy, security, human oversight)
- Innovation highlights (custom tools, agent behaviors, prompt design, etc.)

- Screenshots, logs and evidence
- Reflection and conclusions

## 3. Video Demonstration (5–10 minutes)

- Walkthrough of the input data, model/RAG/agent pipeline, sample interactions, and results.
- Real-time demonstration of system behavior, including grounded responses with citations.

## 4. Presentation, Demonstration and Technical Q&A Duration

| Segment | Duration | Evaluator Focus |
|---|---|---|
| Presentation | 10 minutes | Problem, approach, architecture, results, Responsible AI and limitations |
| Working Demonstration | 10 minutes | Live, end-to-end run of the submitted version with grounded outputs |
| Technical Q&A | 8 minutes | Ownership, design choices, metrics and limitations; questions to each member |
| Evaluator Transition | 2 minutes | Completion of scores and remarks |
| **Total** | **30 minutes** | |

- Apply the same time allocation to every student/team.
- Demonstrate only the submitted version; internet or external API dependencies must be declared in advance.
- If the live demonstration fails, evaluators may use the submitted video as fallback evidence.

## 5. Academic Integrity and AI-Tool Usage Declaration

Every student signs both declarations below; submissions without signed declarations are not evaluated.

- Academic integrity: the submission is original work; all external code, datasets, models and documents are cited; results are not fabricated; individual contributions are accurately stated.
- AI-tool usage: each AI tool or pre-trained model used (including for coding assistance) is declared with its purpose, the artifact affected, and how the output was verified. Students remain accountable for correctness and must be able to explain all submitted work.

| AI Tool / Model | Purpose | Artifact Affected | Verified By Student (Y/N) |
|---|---|---|---|
| | | | |
| | | | |